A PROJECT TITLE

ON

Prompt Versioning and Regression Testing using LLM-as-a-Judge

Project report submitted in partial fulfilment of the requirements for the Course

GENERATIVE AI

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Name : R.C. Sai Sharan Roll no : 2520030128

Name : Kompelli Saanvi Roll no : 2520030083

Under the Esteemed guidance of

Dr. Shaik Asif

CSE

Department of computer science



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

Name of Title : Prompt Versioning and Regression Testing using LLM-as-a-Judge

1. Abstract:

Large Language Models (LLMs) such as ChatGPT and Gemini are widely used for generating text, answering questions, and solving different tasks. The quality of their responses mainly depends on the prompts provided by users. Even small changes in prompts can affect the accuracy and quality of AI-generated responses. Therefore, managing prompt changes and checking their performance is important.

This project proposes a system for Prompt Versioning and Regression Testing using LLM-as-a-Judge. The system maintains different versions of prompts and allows users to compare their performance. Whenever a prompt is modified, regression testing is performed using predefined test inputs to check whether the new version improves or reduces the response quality. An LLM-as-a-Judge approach is used to evaluate the generated responses based on accuracy, relevance, clarity, and instruction following. The system compares the evaluation results of different prompt versions and identifies performance changes. The project aims to make prompt management easier, improve response consistency, and reduce unexpected errors caused by prompt modifications. It will be useful for developers and researchers working with Large Language Model applications.

2. INTRODUCTION:

Large Language Models have become an important part of modern applications such as chatbots, virtual assistants, content generation systems, and question-answering platforms. These models generate responses based on instructions called prompts. The quality of the generated output depends greatly on how these prompts are written and modified.

When developers update a prompt to improve the output, the new version may sometimes produce unexpected results or reduce the quality of previously correct responses. Managing multiple prompt versions and checking their performance manually can be difficult and time-consuming. Therefore, a systematic approach is needed to maintain prompt versions and test their changes.

This project introduces a Prompt Versioning and Regression Testing system using LLM-as-a-Judge. It stores different prompt versions, tests them using predefined inputs, and evaluates the generated responses using another Large Language Model. The evaluation is based on parameters such as accuracy, relevance, clarity, and completeness. The system compares the results of different versions and helps identify the best-performing prompt. This approach aims to improve the reliability, consistency, and quality of AI-generated responses.

3. SOFTWARE REQUIREMENTS:

The proposed system will be developed using Python-based tools and libraries for prompt management, automated testing, LLM evaluation, and performance comparison. An LLM API such as Google Gemini API or OpenAI API will be used to generate and evaluate responses. Streamlit can be used to develop the user interface, while Python will handle prompt versioning, regression testing, and evaluation results.

Software Requirements	Hardware Requirements
Operating System: Windows 10/11 or Ubuntu	Processor: Intel Core i5 or higher
Python 3.10 or above	RAM: Minimum 8 GB
Jupyter Notebook / Google Collab	Internet Connection
Streamlit	Storage: Minimum 10 GB free space
Google Gemini API / OpenAI API	Keyboard and Mouse
Pandas, NumPy	Display Monitor
Git and GitHub	Stable Internet Connection

Signature of the Guide Course Instructor